

nexchat sql

Let me read all the current backend files first before converting. Good, I have the full picture. Now let me install Sequelize and build the SQL conversion: Good. Now write the SQL database module with Sequelize: Now write the full SQL server index: Here's **nexchat-sql.zip** — the full SQL backend. Here's exactly what changed and how it works:

What replaced what

MongoDB / Mongoose	SQL / Sequelize
<code>mongoose.connect()</code>	<code>sequelize.authenticate()</code> + <code>sequelize.sync()</code>
<code>mongoose.Schema</code>	<code>sequelize.define()</code> with <code>DataTypes</code>
<code>pre('save')</code> hook	<code>beforeCreate</code> / <code>beforeUpdate</code> hooks
<code>Room.findOne({ \$or: [...] })</code>	<code>Room.findAll({ where: { [Op.or]: [...] } })</code>
<code>members: [String]</code> array on Room	Separate <code>RoomMember</code> join table
<code>reactions: Map<emoji, String[]></code> on Message	Separate <code>Reaction</code> table with unique constraint
<code>doc.toObject()</code>	<code>instance.get({ plain: true })</code>
ObjectId <code>_id</code>	UUID <code>id</code> (aliased as <code>_id</code> for frontend compatibility)
<code>User.countDocuments()</code>	<code>User.count()</code>

Three dialects — one `.env` switch

```
# SQLite (zero config, local file)
DB_DIALECT=sqlite
SQLITE_PATH=./nexchat.db

# PostgreSQL (Render, Neon, Supabase)
DB_DIALECT=postgres
DATABASE_URL=postgres://user:pass@host:5432/dbname
DB_SSL=true

# MySQL (PlanetScale, Railway)
```

```
DB_DIALECT=mysql
DATABASE_URL=mysql://user:pass@host:3306/dbname
```

Quick start

```
unzip nexchat-sql.zip && cd nexchat-sql
npm install

# SQLite – needs the driver
npm install sqlite3      # or: npm install better-sqlite3

cp .env.example .env     # set DB_DIALECT=sqlite for local
dev
npm start
# Tables are auto-created on first run via sequelize.sync({
alter: true })
```

New file — `server/db.js`

This is the only new file. It contains all Sequelize model definitions, associations, hooks, and query helper functions (`findRoomsForUser` , `addMember` , `removeMember` , `findMessages` , `addReaction`) that match the same interface the rest of the server expects. The `server/index.js` calls these helpers and never writes raw SQL — switching dialects only requires changing `DB_DIALECT` in `.env` .